

Progress Report 2

Progress Report 2 - OneFitPass Canada

CSIS 3275 – 070, Software Engineering

Team Members:

Mikhail Senatorov

Jana Paul

Omkar Patel - 300374615

Instructor: Caesar Jude Clemente

Content

Progress Report 2	1
1. Introduction	2
2. Design Goals	3
2.1 Performance-Oriented Architecture	3
2.2 Scalability and Growth Readiness	4
2.3 Security and Compliance Design	5

2.4 Reliability and Fault Tolerance	6
2.5 Maintainability and Configurability	6
3. System Architecture	7
3.1 Architectural Pattern	7
3.2 Component Interaction Overview	8
3.3 Infrastructure Overview	9
4. Hardware and Software Configuration	11
4.1 Server Configuration	11
4.2 Client Configuration	12
5. Database Design	12
5.1 Core Tables	12
5.2 Normalization Justification	15
6. Interface Design	15
7. Revised UML Diagrams	19
7.1 Class Diagram	19
7.2 Sequence Diagram – Check-In Process	21
8. Design Phase KPIs	23
9. Design Phase Conclusion	23
References	24

1. Introduction

Progress Report 2 advances the OneFitPass Canada project from the requirement analysis phase detailed in Progress Report 1 to a comprehensive system design. While the initial report established the business case, problem statement, revenue model, functional and non-functional requirements, feasibility studies, and KPIs for requirements validation, this report focuses on translating those elements into a robust architectural blueprint.

This document outlines the design goals explicitly mapped to non-functional requirements, the overall system architecture including component interactions and infrastructure, hardware and software configurations, database schema with normalization rationale, interface design

principles (with placeholders for prototypes), revised UML diagrams reflecting the solution domain, and KPIs specific to the design phase.

The objective is to confirm that the proposed solution is not only feasible but also architecturally sound, scalable, secure, reliable, and maintainable. By grounding the design in the validated requirements, we ensure alignment with user needs, business objectives, and technical constraints. This phase positions the project for efficient implementation, minimizing risks and facilitating future expansions.

2. Design Goals

Design goals serve as the bridge between non-functional requirements and concrete architectural decisions. Each goal is directly mapped to the quality attributes defined in Progress Report 1, ensuring that the system design prioritizes performance, scalability, security, reliability, and maintainability. These goals guide the selection of technologies, patterns, and strategies to deliver a high-quality platform.

2.1 Performance-Oriented Architecture

Mapped Non-Functional Requirements:

- Average loading time for core features (login, check-in) shall not exceed 2 seconds under normal load.
- Check-ins shall be server-validated in under 1 second.
- Backend APIs shall maintain response times of less than 500 milliseconds for standard queries.
- The system shall handle peak usage periods without noticeable performance degradation.

Design Strategy: To meet these performance benchmarks, the architecture incorporates optimized data handling and efficient processing. Key elements include:

- Adoption of RESTful API design with lightweight JSON payloads to minimize data transfer overhead.
- Utilization of indexed PostgreSQL queries for rapid data retrieval in high-frequency operations, such as user authentication and gym searches.

- Implementation of Redis in-memory caching for frequently accessed data, including gym listings, subscription statuses, and user profiles, reducing database load.
- Stateless backend services using Node.js, enabling quick scaling and load balancing without session affinity.

This strategy ensures sub-second response times and graceful handling of peak loads, such as during morning or evening gym rushes. Performance testing will validate these elements against simulated user traffic.

2.2 Scalability and Growth Readiness

Mapped Non-Functional Requirements:

- The system architecture shall support at least 10,000 concurrent active users.
- The platform shall scale from a regional MVP (minimum viable product) to nationwide operation across Canada without fundamental architectural redesign.
- The platform shall be capable of managing 1,000+ partner gyms and 50,000+ registered users.

Design Strategy: Scalability is achieved through a cloud-native approach that allows elastic resource allocation. Specific measures include:

- Deployment on AWS with auto-scaling groups to dynamically adjust compute resources based on traffic.
- Load balancing via AWS Elastic Load Balancer (ELB) to distribute requests evenly across instances.
- Containerization using Docker for services, facilitating easy replication and deployment in Kubernetes for advanced orchestration if needed.
- Modular monolith structure initially, with clear service boundaries (e.g., authentication, payments, check-ins) to enable future migration to microservices without redesign.
- Database sharding preparedness, with PostgreSQL partitioned by user or gym ID for horizontal scaling.

This design supports growth from an MVP focused on Metro Vancouver to national coverage, handling increased user bases and gym partnerships seamlessly. Projections indicate the system can scale to 50,000 users with minimal additional infrastructure costs.

2.3 Security and Compliance Design

Mapped Non-Functional Requirements:

- The platform shall comply with Canadian data protection legislation (PIPEDA).
- Payment processing shall be handled exclusively through PCI-DSS-compliant payment providers.
- User and partner accounts shall be protected using secure authentication mechanisms.
- QR codes shall be dynamic to prevent misuse or credential sharing.
- All data transmission shall be encrypted using TLS and HTTPS.

Design Strategy: Security is integrated as a core principle, with compliance baked into every layer. Key implementations include:

- Enforcement of TLS 1.3 encryption for all API communications and data in transit.
- Use of Stripe for payment processing, ensuring PCI-DSS Level 1 compliance and tokenization to avoid storing sensitive card data.
- OAuth 2.0 and OpenID Connect protocols via Auth0 or AWS Cognito for secure authentication, supporting social logins and JWT tokens.
- Role-Based Access Control (RBAC) to restrict features based on user roles (e.g., gym partners cannot access admin tools).
- Mandatory Multi-Factor Authentication (MFA) for gym partners and administrators to protect sensitive operations.
- Dynamic QR codes generated with time-limited (5-minute validity) and single-use encrypted tokens to prevent replay attacks or sharing.
- Comprehensive audit logging of all security-relevant events, including check-ins, payments, and login attempts, stored immutably for compliance audits.

This approach mitigates risks such as data breaches and fraud, ensuring PIPEDA compliance through data residency in Canadian AWS regions and user consent mechanisms.

2.4 Reliability and Fault Tolerance

Mapped Non-Functional Requirements:

- The platform shall achieve a minimum monthly availability of 99.5%.
- Check-ins, payments, and payouts shall be transaction-safe, ensuring no data loss.
- The system shall support a limited offline check-in functionality.
- Automated backups of data shall be performed at least daily.

Design Strategy: Reliability is ensured through redundant systems and failover mechanisms:

- Use of PostgreSQL with ACID (Atomicity, Consistency, Isolation, Durability) compliance for transaction-safe operations in critical flows like check-ins and payments.
- Automated daily backups via AWS RDS snapshots, with point-in-time recovery enabled.
- Multi-availability zone deployment on AWS to handle regional outages.
- Retry logic and circuit breakers in the backend for transient failures in payment processing or external integrations.
- Event logging and monitoring with tools like AWS CloudWatch for proactive issue detection.
- Limited offline mode for check-ins using encrypted local storage on mobile devices, syncing when connectivity is restored.

These measures guarantee high uptime and data integrity, with recovery time objectives (RTO) under 1 hour for major incidents.

2.5 Maintainability and Configurability

Mapped Non-Functional Requirements:

- The codebase shall be modular, well-documented, and version-controlled.

- Subscription plans, pricing, and commission rates shall be configurable through the administrative role without requiring any modification to the system code.
- Monitoring and logging mechanisms shall be implemented to enable early detection of system issues.
- System updates and maintenance shall be deployable with minimal service interruption.

Design Strategy: Maintainability focuses on ease of updates and long-term sustainability:

- Modular codebase structured with the Controller-Service-Repository pattern in Node.js, promoting separation of concerns.
- Git-based version control with branching strategies (e.g., GitFlow) for collaborative development.
- Centralized configuration management using environment variables and database-stored settings for plans and pricing, editable via admin dashboard.
- Comprehensive documentation including API specs (Swagger/OpenAPI) and inline code comments.
- Blue-green deployments on AWS for zero-downtime updates.
- Integrated monitoring with Prometheus and Grafana for metrics, logs, and alerts.

This ensures the system can evolve without significant rework, supporting agile iterations and cost-effective maintenance.

3. System Architecture

The system architecture adopts a modern, layered approach to ensure separation of concerns, scalability, and ease of integration. It builds upon the technology stack outlined in the feasibility study, refining it for implementation.

3.1 Architectural Pattern

The system follows a three-tier client-server architecture, which decouples the user interface, business logic, and data storage for better maintainability and scalability.

Presentation Layer:

- React.js for the web application, providing a responsive and interactive user experience.
- React Native for the mobile app, enabling cross-platform development for iOS and Android with shared codebases.

Application Layer:

- Node.js with NestJS framework for structured, testable backend code.
- RESTful APIs for communication between frontend and backend, using JSON for data exchange.
- Middleware for authentication, logging, and rate limiting.

Data Layer:

- PostgreSQL as the primary relational database for structured data.
- Redis for caching and session management to enhance performance.

This tiered structure facilitates independent scaling of each layer and supports future enhancements, such as adding GraphQL for more flexible queries.

3.2 Component Interaction Overview

The architecture emphasizes efficient interactions between components to minimize latency and ensure data consistency. Key flows are illustrated below.

User Check-In Flow:

1. The user opens the mobile app and requests a QR code.
2. The app sends an authenticated request to the backend API.
3. The backend verifies the user's subscription status via the database.
4. A dynamic QR code with an encrypted token is generated and returned.
5. The gym partner scans the QR using their app or device.
6. The backend validates: a. Active subscription. b. Remaining visit limit. c. Fraud indicators (e.g., geolocation match).

- 7. The check-in is recorded transactionally in the database.
- 8. Confirmations are pushed to both user and gym via real-time notifications (e.g., WebSockets).

All interactions occur over secure HTTPS channels, with error handling for network failures or invalid states.

Subscription Purchase Flow:

- 1. User selects a plan in the app.
- 2. Backend processes payment via Stripe integration.
- 3. Upon success, subscription record is created in the database.
- 4. User receives immediate access updates.

These flows ensure atomicity and reliability through database transactions.

3.3 Infrastructure Overview

The infrastructure is cloud-based for flexibility and cost-efficiency, leveraging AWS services compliant with Canadian data residency requirements.

Component	Technology
Cloud Provider	AWS (ca-central-1 region)
Compute	EC2 instances (auto-scaling)
Database	RDS PostgreSQL
Cache	ElastiCache Redis
Storage	S3 for user uploads (e.g., gym photos)
CDN	CloudFront for asset distribution
Payment Gateway	Stripe
Authentication	Auth0 or Cognito

Deployment is managed via AWS CodePipeline for CI/CD, with Nginx as a reverse proxy for load balancing and SSL termination. Monitoring is handled by CloudWatch, ensuring proactive alerting.

3.4 Advanced Check-In Validation & Fraud Prevention Architecture

The QR-based check-in mechanism represents the most security-critical and performance-sensitive component of the OneFitPass platform. To ensure transactional integrity, fraud resistance, and scalability, the check-in system incorporates multiple validation layers beyond simple token verification.

1. Token Generation & Expiry

Each QR code contains:

- A time-limited JWT (JSON Web Token)
- Encrypted user ID
- Encrypted subscription ID
- Token expiration timestamp (5 minutes)
- Unique nonce (single-use identifier)

Tokens are generated server-side and digitally signed to prevent tampering. Expired or reused tokens are automatically rejected.

2. Multi-Layer Validation Process

Upon scanning, the backend performs:

1. Token signature verification
2. Expiration timestamp validation
3. Subscription status check
4. Visit-limit check
5. Duplicate check-in prevention (same gym within configurable cooldown period, e.g., 30 minutes)
6. Optional geolocation proximity check (within 100 meters radius)

All validations are executed within a database transaction to prevent race conditions.

3. Concurrency & Race Condition Handling

To prevent double check-ins from rapid repeated scans:

- A database-level unique constraint is applied on (user_id, gym_id, timestamp window).
- Transactions use SERIALIZABLE isolation level for critical validation.
- Redis is used as a short-term lock (distributed lock mechanism) during validation.

This ensures that only one valid check-in can occur per authorized attempt.

4. Fraud Detection Indicators

The system logs suspicious activity patterns, including:

- Multiple gym check-ins within unrealistic time intervals
- Token reuse attempts
- GPS mismatch between user and gym location
- Excessive failed scan attempts

Flagged activity triggers admin review workflows.

5. Performance Optimization

To maintain sub-1-second validation:

- Active subscription data is cached in Redis
- Gym verification status is cached
- Only final check-in record is written to PostgreSQL

Expected validation time: 200–500ms under normal load.

6. Data Logging & Audit Trail

Each check-in generates:

- Immutable event log entry
- Timestamp
- Validation result
- Fraud indicator flags (if applicable)

This supports compliance audits and dispute resolution.

4. Hardware and Software Configuration

Configurations are optimized for the MVP phase, with provisions for scaling. All choices align with cost estimates from the feasibility study.

4.1 Server Configuration

- **Instance Type:** EC2 t3.medium (2 vCPUs, 4 GiB RAM) for initial MVP, scalable to larger instances.
- **Operating System:** Ubuntu 22.04 LTS for stability and security updates.
- **Web Server:** Nginx for handling HTTP requests and reverse proxying to Node.js.
- **Runtime:** Node.js v20+ with pm2 process manager for application clustering.
- **Database:** PostgreSQL 14+, configured with connection pooling (pgBouncer).

Security configurations include firewall rules (AWS Security Groups) limiting access to ports 80/443, and automated backups.

4.2 Client Configuration

User Devices:

- **Mobile:** Android 10+ and iOS 14+ for broad compatibility.
- **Web:** Supported on modern browsers including Chrome 100+, Edge 100+, Safari 15+, Firefox 100+.

Gym Partner Devices:

- Any tablet or smartphone with a camera for QR scanning.
- Requires stable internet (Wi-Fi or cellular) for real-time validation.

No high-end hardware is required, ensuring accessibility for small gyms. The app is optimized for low-bandwidth environments.

5. Database Design

The database is designed as a relational model in PostgreSQL, normalized to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity. It supports core entities like users, subscriptions, gyms, and transactions.

5.1 Core Tables

Users Table: Stores user account information.

Column	Type	Constraints	Description
user_id	SERIAL	PRIMARY KEY	Unique identifier
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Email address for login
password_hash	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM('user', 'partner', 'admin')	NOT NULL	User role for RBAC
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

Plans Table: Defines subscription tiers.

Column	Type	Constraints	Description
plan_id	SERIAL	PRIMARY KEY	Unique plan ID
name	VARCHAR(50)	NOT NULL	Plan name (e.g., 'Basic')
monthly_price	DECIMAL(10,2)	NOT NULL	Monthly cost
visit_limit	INTEGER		Max visits per month (null for unlimited)
commission_rate	DECIMAL(5,2)	NOT NULL	Platform commission percentage

Subscriptions Table: Tracks active user subscriptions.

Column	Type	Constraints	Description
subscription_id	SERIAL	PRIMARY KEY	Unique subscription ID
user_id	INTEGER	FOREIGN KEY (Users)	Linked user

plan_id	INTEGER	FOREIGN KEY (Plans)	Linked plan
start_date	DATE	NOT NULL	Subscription start
end_date	DATE		Subscription end (for fixed-term)
status	ENUM('active', 'paused', 'cancelled')	NOT NULL	Current status

Gyms Table: Stores partner gym details.

Column	Type	Constraints	Description
gym_id	SERIAL	PRIMARY KEY	Unique gym ID
name	VARCHAR(100)	NOT NULL	Gym name
address	VARCHAR(255)	NOT NULL	Physical address
verified_status	BOOLEAN	DEFAULT FALSE	Verification flag
opening_hours	JSONB		Schedule in JSON format

CheckIns Table: Logs user visits.

Column	Type	Constraints	Description
checkin_id	SERIAL	PRIMARY KEY	Unique check- in ID
user_id	INTEGER	FOREIGN KEY (Users)	User who checked in
gym_id	INTEGER	FOREIGN KEY (Gyms)	Gym visited
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Check-in time

validation_status	ENUM('valid', 'invalid')	NOT NULL	Validation result
-------------------	--------------------------	----------	-------------------

Payouts Table: Manages gym payouts.

Column	Type	Constraints	Description
payout_id	SERIAL	PRIMARY KEY	Unique payout ID
gym_id	INTEGER	FOREIGN KEY (Gyms)	Linked gym
total_amount	DECIMAL(10,2)	NOT NULL	Payout amount
payout_month	DATE	NOT NULL	Month of payout
status	ENUM('pending', 'paid')	NOT NULL	Payout status

These tables form the core data model, with foreign keys enforcing referential integrity.

5.2 Normalization Justification

The schema adheres to normalization principles to reduce data redundancy and anomalies:

- **1NF (First Normal Form):** No repeating groups; all attributes are atomic (e.g., opening_hours stored as JSONB for structured data without multi-valued fields).
- **2NF (Second Normal Form):** No partial dependencies on composite keys; all non-key attributes depend on the entire primary key (no composite keys used in core tables).
- **3NF (Third Normal Form):** No transitive dependencies; attributes depend only on the primary key (e.g., subscription status is directly tied to subscription_id, not derived from other attributes).

This normalization minimizes update anomalies (e.g., changing a plan's price affects only the Plans table) and supports efficient querying. Denormalization is avoided to maintain data consistency, with caching handling performance needs.

6. Interface Design

Interface design prioritizes usability, aligning with non-functional requirements for intuitive navigation and minimal steps. Prototypes (low-fidelity wireframes created in Balsamiq) will be inserted here upon completion.

6.1 Key principles:

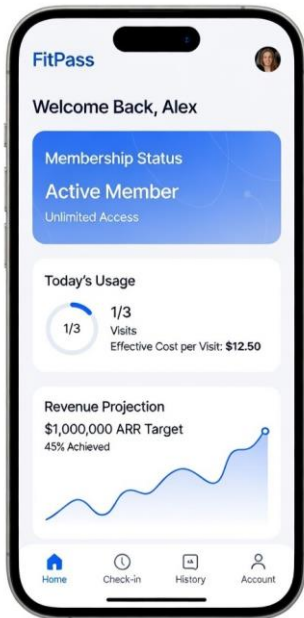
- **Mobile-First Layout:** All screens optimized for smartphones, with responsive adaptations for tablets and desktops.
- **Maximum 3 Steps for Core Actions:** E.g., check-in: Open app → Generate QR → Scan.
- **Clear Visual Hierarchy:** Use of color coding (e.g., green for active subscriptions), icons, and whitespace to guide user focus.
- **Minimal Cognitive Load:** Simplified forms, auto-complete for searches, and contextual help.
- **Action-Oriented Error Messages:** E.g., "Subscription expired – Renew now?" with direct links.

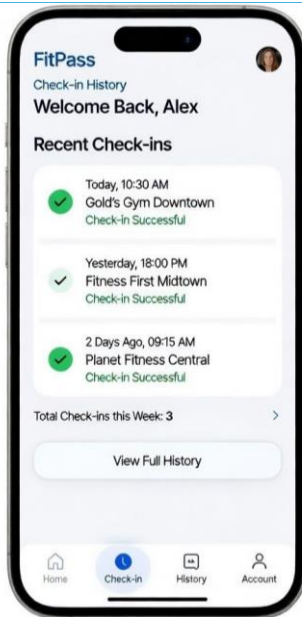
Prototype Overviews:

- **User Dashboard Layout:** Displays subscription status, remaining visits, recent check-ins, and gym recommendations.
- **Gym Search Interface:** Map-based search with filters for location, hours, and amenities.
- **QR Check-In Screen:** Simple button to generate QR, with real-time validity timer.
- **Partner Dashboard:** Analytics on visits, payouts, and visitor stats.
- **Admin Subscription Management Panel:** Tools to configure plans, view users, and resolve disputes.

These designs ensure accessibility and compliance with WCAG standards.

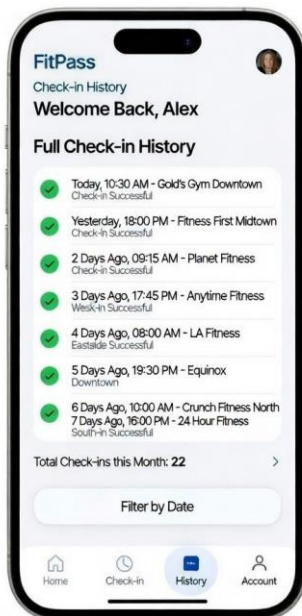
6.2 App Interface Overview

Screenshot	Description
	Home Screen: FitPass app dashboard showing "Welcome Back, Alex", Membership Status card with "Active Member" and "Unlimited Access", Today's Usage circle with 1/3 Visits and Effective Cost per Visit \$12.50, Revenue Projection graph targeting \$1,000,000 ARR at 45% Achieved, bottom nav with Home, Check-in, History, Account icons. This is the main dashboard for users. It displays subscription status, daily visit tracking to show value (like cost per visit), and revenue projections for motivation or admin view. Keeps everything quick-load, mobile-first, with graphs for visual punch. Perfect for checking if your pass is active before hitting the gym.
	Check-in Screen: FitPass "Check-in History" showing "Welcome Back, Alex", Recent Check-ins list with Today's 10:30 AM at Gold's Gym Downtown



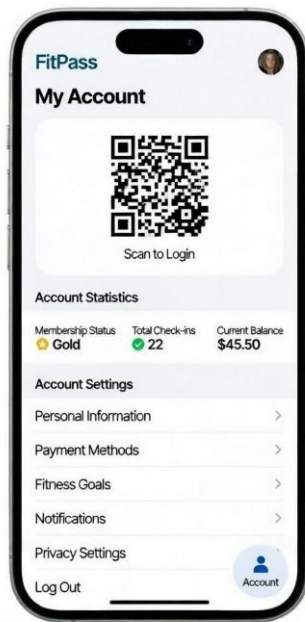
(successful), Yesterday's 18:00 PM at Fitness First Midtown (successful), 2 Days Ago 09:15 AM at Planet Fitness Central (successful), Total Check-ins this Week: 3, "View Full History" button, bottom nav with Check-in highlighted.

Here's the check-in tab focusing on recent visits. Lists last few gym entries with timestamps, locations, and success status (green checks). Shows weekly total to track usage against limits. Ties into QR validation from report – dynamic codes prevent fraud, geolocation checks. Good for users to see if visits counted right.



History Screen: FitPass "Check-in History" showing "Welcome Back, Alex", Full Check-in History list with Today at Gold's Gym, Yesterday at Fitness First, 2-7 Days Ago at various gyms like Planet Fitness, Anytime Fitness, LA Fitness, Equinox, Crunch, 24 Hour Fitness (all successful), Total Check-ins this Month: 22, "Filter by Date" button, bottom nav with History highlighted.

Full history view for deeper dive. Scrolls through all monthly visits with dates, times, gym names, and success icons. Monthly total helps with billing or limits (e.g., visit caps from plans). From report: logged in CheckIns table with validation_status. Users can filter for disputes or patterns, like favorite gyms.



Account Screen: FitPass "My Account" showing QR code for "Scan to Login", Account Statistics with Membership Status "Gold" (star), Total Check-ins 22 (check), Current Balance \$45.50, Account Settings list: Personal Information, Payment Methods, Fitness Goals, Notifications, Privacy Settings, Log Out, bottom nav with Account highlighted.

Account tab with QR for quick gym entry (dynamic, time-limited as per security reqs). Stats show membership tier, visit count, balance for payouts or credits. Settings for personal data, payments (Stripe integration), goals tracking, notifs, privacy (PIPEDA compliant). Log out at bottom. All tied to Users/Subscriptions tables from DB design.

7. Revised UML Diagrams

UML diagrams have been revised to reflect the solution domain, incorporating design decisions like specific classes and interactions.

7.1 Class Diagram

The class diagram models core entities and relationships:

Core Classes:

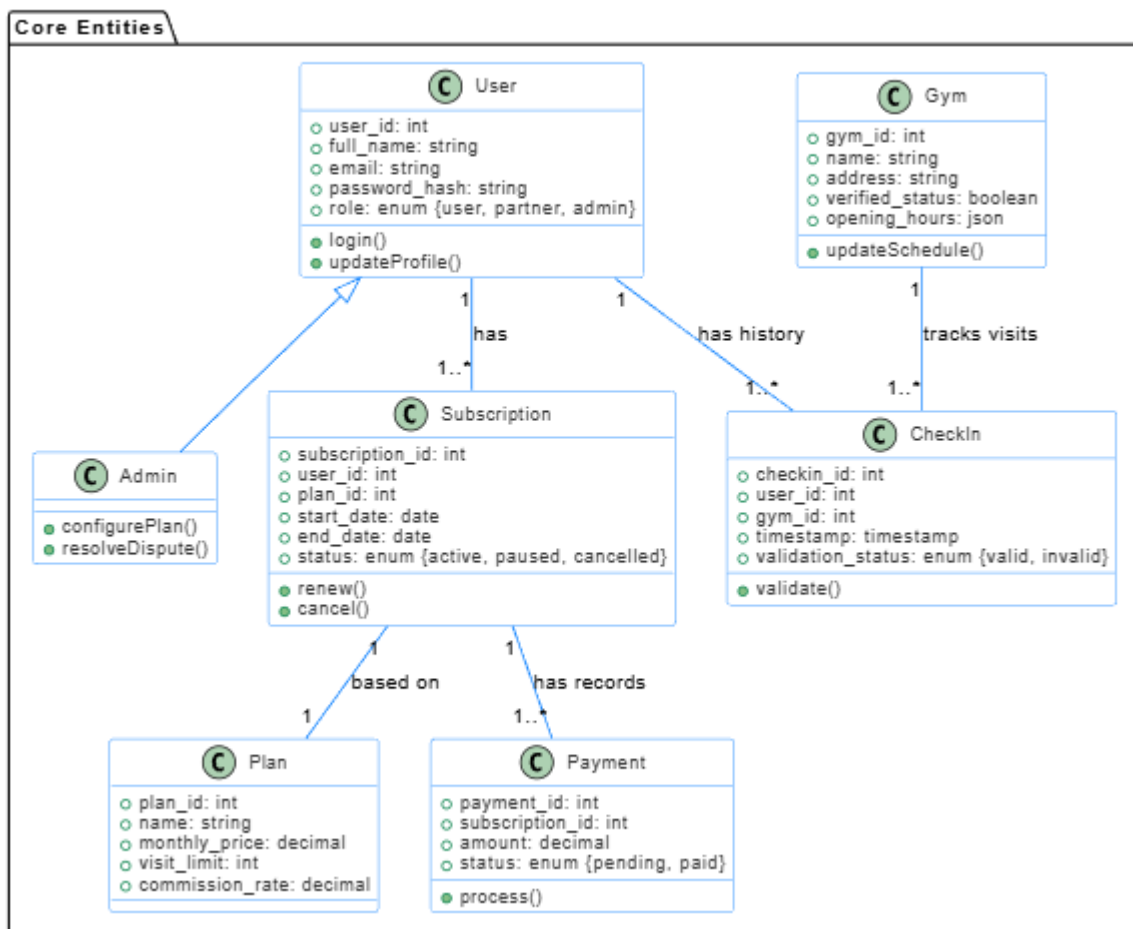
- User: Attributes (user_id, full_name, email, password_hash, role); Methods (login, updateProfile).
- Subscription: Attributes (subscription_id, user_id, plan_id, start_date, end_date, status); Methods (renew, cancel).
- Plan: Attributes (plan_id, name, monthly_price, visit_limit, commission_rate).
- Gym: Attributes (gym_id, name, address, verified_status, opening_hours); Methods (updateSchedule).

- CheckIn: Attributes (checkin_id, user_id, gym_id, timestamp, validation_status); Methods (validate).
- Payment: Attributes (payment_id, subscription_id, amount, status); Methods (process).
- Admin: Extends User; Methods (configurePlan, resolveDispute).

Relationships:

- User 1..* Subscription (a user can have multiple subscriptions over time).
- Subscription 1 Plan (each subscription is based on one plan).
- User 1..* CheckIn (users have visit history).
- Gym 1..* CheckIn (gyms track visits).
- Subscription 1..* Payment (subscriptions have payment records).

Class Diagram for OneFitPass Canada



This diagram ensures object-oriented design with inheritance (Admin from User) and associations. Key classes include:

User as the base user entity with RBAC roles for authentication and profiles;

Admin extending User for admin privileges like configuring plans and resolving disputes;

Subscription managing user plans over time with methods for renewal and cancellation;

Plan defining tiers with limits and pricing;

Gym representing partner gyms with verification and schedule updates;

CheckIn logging visits with validation;

Payment handling transactions via Stripe.

Relationships highlight multiplicity: a User can have multiple Subscriptions and CheckIns, while Subscriptions link to one **Plan** and multiple Payments, and CheckIns associate with Users and Gyms.

This structure supports scalability, data integrity via foreign keys, and modular maintenance, aligning with the database design in Section 5.

7.2 Sequence Diagram – Check-In Process

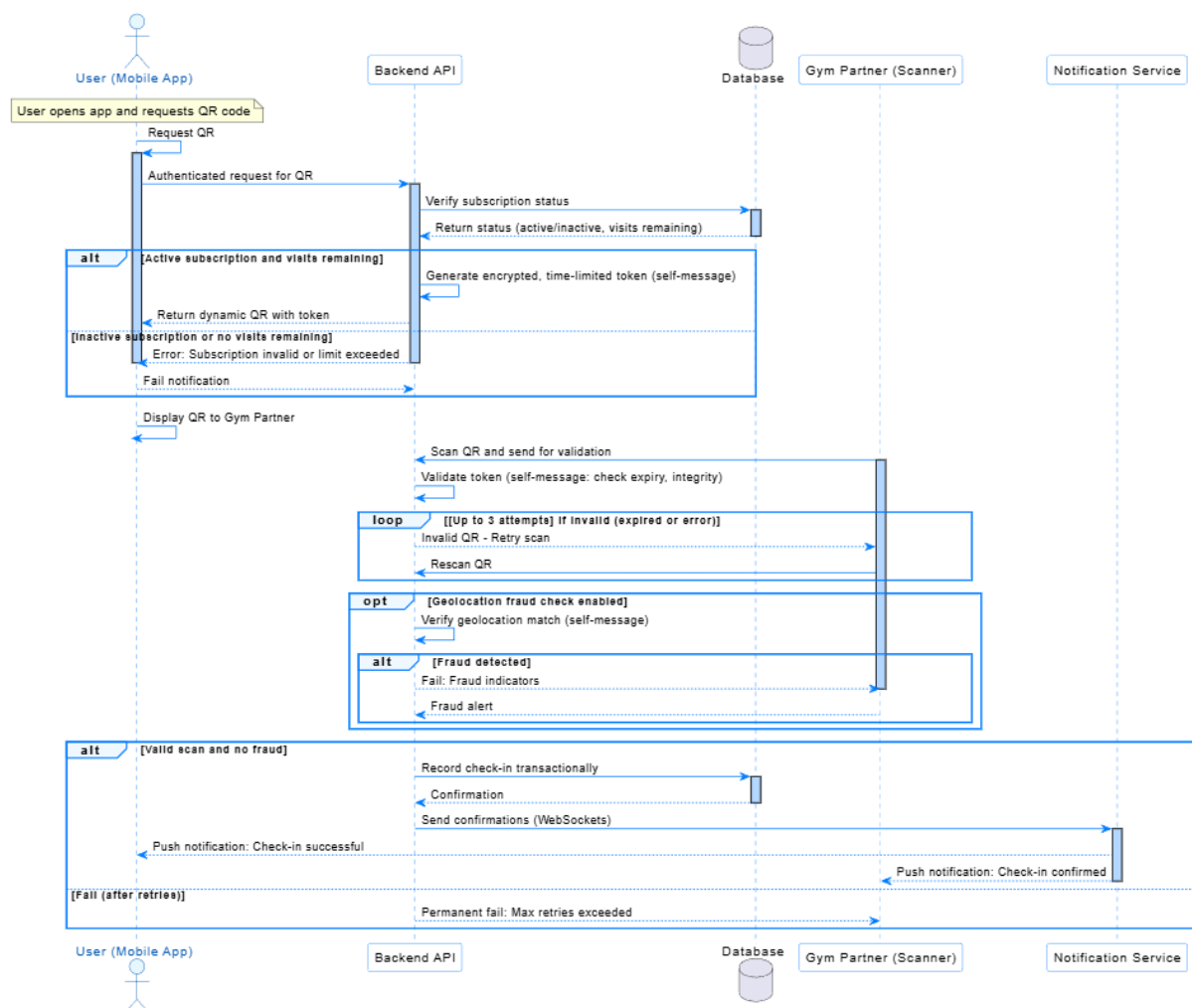
Sequence Diagram for User Check-In Process

This sequence diagram illustrates the check-in flow in the OneFitPass Canada platform, starting from the user requesting a QR code in the mobile app. It demonstrates interactions between the User (via Mobile App), Backend API, Database, Gym Partner (Scanner), and Notification Service. Key elements include:

- **Conditional alternatives (alt):** Handles success (active subscription, available visits, no fraud) vs. failure (inactive subscription or exceeded visit limit).
- **Loop:** Allows up to 3 retry attempts for invalid QR scans (e.g., expired token or scan error).
- **Optional part (opt):** Geolocation fraud check, performed only if enabled in the system configuration.

- **Self-messages:** Internal validations within the Backend API, such as token generation and limit checks.
- **Activation bars:** Show processing lifespans for services like Backend API and Database to highlight concurrency and resource usage.

The diagram ensures transactional integrity (e.g., check-in recorded only on success) and real-time notifications via WebSockets, aligning with non-functional requirements for security, reliability, and performance.



Actors: User, Mobile App, Backend, Database, Gym Partner.

Steps:

1. User → Mobile App: Request QR.
2. Mobile App → Backend: Generate token (authenticated).

3. Backend → Database: Verify subscription.
4. Database → Backend: Return status.
5. Backend → Mobile App: Return QR with token.
6. Gym Partner → Backend: Scan QR (validate token).
7. Backend → Database: Check limits and record check-in.
8. Database → Backend: Confirmation.
9. Backend → Mobile App/Gym Partner: Return success notification.

This sequence emphasizes synchronous validation for real-time feedback and error handling (e.g., invalid token returns failure).

8. Design Phase KPIs

Key Performance Indicators (KPIs) for the design phase measure the architecture's alignment with requirements, ensuring readiness for implementation.

KPI	Target
API response time	<500ms
Check-in validation	<1 sec
System uptime	≥99.5%
Concurrent users supported	10,000+
Database normalization	3NF
Security requirement coverage	100%
Code modularity (modules defined)	100%
Configuration without code changes	100%

These KPIs will be validated through design reviews and prototypes.

9. Design Phase Conclusion

Progress Report 2 demonstrates that OneFitPass Canada is architecturally sound, technically achievable, secure, scalable, and aligned with the requirements from Report 1.

The system design:

- Maps non-functional requirements directly to architectural decisions.
- Defines a cloud-based scalable infrastructure.
- Establishes secure authentication and payment handling.
- Implements a normalized relational database.
- Provides maintainable modular architecture.
- Includes revised UML for clear implementation guidance.

With these elements in place, the project is now ready to proceed into implementation, UI prototyping, and testing phases. Risks are mitigated through proven technologies and modular design, positioning OneFitPass for successful launch and growth.

References

- Amazon Web Services. (2025). AWS Documentation. <https://docs.aws.amazon.com/>
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- Fowler, M. (2018). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Microsoft. (2025). Azure Architecture Center. <https://docs.microsoft.com/en-us/azure/architecture/> (adapted for AWS).
- PostgreSQL Documentation. (2025). <https://www.postgresql.org/docs/>
- NestJS Documentation. (2025). <https://docs.nestjs.com/>
- React Native Documentation. (2025). <https://reactnative.dev/docs/>